

[← Back to CCDV-F Sample Tests](#)

Great Job!

Eval, Testing, and Debugging Practice Test Complete

10/10

Correct

100%

Score

6:43

Time



This was a sample test from a limited pool of questions.

Upgrade to Premium for the complete CCDV-F bank with hundreds more questions!

Upgrade to Premium

Get unlimited access to:

- ✓ Unlimited practice tests across all domains
- ✓ Full-length 53-question mock exams with realistic timing
- ✓ Detailed explanations and study guides
- ✓ Progress tracking and performance analytics
- ✓ Smart practice focusing on weak areas

Review Your Answers

1

[→ Login to Clarify](#)

A staging chatbot suddenly fails before returning any assistant message. The trace shows:

```
request_id: req_4821
POST /v1/messages
status: 401
response: {"error": "invalid API key"}
model_response_received: false
retry_count: 3
```

What is the BEST classification of this failure?

A.

A model-output failure caused by an unhelpful response

B.

An integration-layer failure caused before model generation



C.

A schema validation failure in the post-processing layer

D.

A prompt-quality failure caused by unclear instructions

Explanation:

Correct answer (B): The log shows an HTTP 401 and explicitly says no model response was received. That places the first failure boundary before Claude generated output, so the correct diagnosis is an integration-layer issue involving authentication or request setup. Retrying without fixing the API key will not address the root cause.

Why the other options are wrong:

- Option A: A model-output failure may seem plausible because the chatbot produced no useful answer, but the log shows no model response was generated.
- Option C: Schema validation is a post-processing concern, but the request failed at

authentication before any response content could be validated.

- Option D: Prompt quality can affect generated content, but the model was never reached, so prompt changes are not the right diagnosis.

2 

[→ Login to Clarify](#)

A team tests a Claude-powered support reply generator. The failing test log shows:

```
expected: "Your refund will be processed in 5-7 business days."  
actual:   "We'll process your refund within five to seven business days."  
policy_check: passed  
required_fields: passed  
customer_tone: passed
```

The task does not require exact wording. What is the BEST test change?

- A.
Replace the exact-string assertion with semantic acceptance checks



- B.
Increase retry count until the exact sentence is produced

- C.
Fail the response because it differs from the golden text

- D.
Disable automated testing for this generated reply

Explanation:

Correct answer (A): Because the task does not require exact wording and all task-specific checks passed, the test is brittle. LLM output can vary while still satisfying the requirement. A better test validates semantics, required constraints, structured fields, and acceptance criteria instead of a single exact sentence.

Why the other options are wrong:

- Option B: Retries may occasionally reproduce the target wording, but they increase cost and hide a brittle test design rather than fixing it.
- Option C: Golden text can be useful when exact wording matters, but the scenario says equivalent wording is acceptable.
- Option D: Disabling testing avoids false failures but removes regression coverage instead of improving the assertion.

3 

[→ Login to Clarify](#)

A batch summarization worker logs the following for one document:

```
attempt 1: status 429, response "rate limit exceeded"
attempt 2: status 429, response "rate limit exceeded"
attempt 3: scheduled after 200 ms, no jitter
operation: generate summary only, no external side effect
```

What is the BEST recovery strategy?

A.

Stop all retries because every 429 is a permanent request error

B.

Use bounded exponential backoff with jitter for this request



C.

Rewrite the prompt because rate limits indicate poor instructions

D.

Ignore the failed document and mark the summary as complete

Explanation:

Correct answer (B): A rate-limit response is a transient operational failure, and the operation is a generation-only task with no external side effect. The best recovery is a bounded retry policy with exponential backoff and jitter. This avoids hammering the service while still allowing recovery from temporary capacity limits.

Why the other options are wrong:

- Option A: Some errors are permanent, but the scenario identifies a rate-limit response, which is commonly handled as transient with bounded backoff.
- Option C: Prompt wording does not address a rate-limit status because the failure is operational rather than model-output quality.
- Option D: Marking the document complete loses work and hides the failure instead of applying an appropriate transient-error policy.

4

[→ Login to Clarify](#)

A Claude-powered order assistant uses a shipping lookup tool. The trace for a failed request shows:

1. API status: 200
2. assistant tool_use: get_shipping_quote {"zip":"94107","weight_kg":2}
3. app dispatch: get_shipping_quote
4. tool result: HTTP 500 from carrier API
5. assistant continuation: not requested

Where did the failure first occur?

- A.
During Claude API request construction
- B.
During model generation before tool selection
- C.
During external tool execution after dispatch

- D.
During final response schema validation

Explanation:

Correct answer (C): The trace shows that the Claude API request succeeded and the model produced a tool request. The application then dispatched the tool, and the first failure signal is the carrier API HTTP 500. Therefore the failing boundary is external tool execution, not model generation or validation.

Why the other options are wrong:

- Option A: Request construction succeeded because the API returned 200 and a valid tool_use event was produced.
- Option B: The model successfully selected a tool and supplied arguments, so the first

failure is not before tool selection.

- Option D: No final assistant continuation occurred, so final response validation is downstream of the observed failure.

5

[→ Login to Clarify](#)

A product classifier started failing after a backend refactor. The raw Claude output and validator log show:

```
raw_model_content: {"category":"hardware","confidence":0.82}
old_contract: {category:string, confidence:number}
new_validator_error: missing required field "label"
new_contract: {label:string, confidence:number}
prompt_version: unchanged for 3 weeks
```

What is the BEST next diagnostic conclusion?

A.

The model has become less capable and needs a larger model

B.

The prompt is newly ambiguous and should be rewritten first

C.

The validator contract changed without aligning the expected output



D.

The API transport failed and returned malformed response bytes

Explanation:

Correct answer (C): The raw model output still matches the old contract, and the prompt has not changed. The failure appeared after a backend refactor that introduced a validator expecting ``label`` instead of ``category``. The best conclusion is an application contract mismatch, not a sudden model-quality or transport problem.

Why the other options are wrong:

- Option A: A larger model might be considered for quality issues, but the evidence points to a schema contract mismatch introduced by the refactor.
- Option B: The prompt is unchanged, and the model output matches the old expected shape, so prompt ambiguity is not the first conclusion.
- Option D: The raw content is well-formed and available to the validator, so malformed transport bytes are not indicated.

6

[→ Login to Clarify](#)

A Claude workflow drafts an invoice email, calls a payment API tool, then asks Claude to summarize the result. A timeout occurs after the tool call:

```
tool_call: charge_customer {"invoice_id":"INV-77","amount":200}
payment_api: request sent
client_error: timeout waiting for payment_api response
retry_policy: immediate retry of whole workflow
idempotency_key: none
```

What is the BEST debugging-driven mitigation before enabling retries?

A.

Retry the full workflow immediately to maximize completion rate

B.

Change the summary prompt so Claude reports fewer timeouts

C.

Use a cheaper model so retries cost less when failures occur

D.

Add idempotency or status checks around the payment operation



Explanation:

Correct answer (D): The timeout happened after a side-effecting payment request may have been sent. Retrying the whole workflow without an idempotency key or status check can duplicate the charge. The correct mitigation is to make recovery idempotency-aware before retrying an ambiguous external action.

Why the other options are wrong:

- Option A: Immediate full-workflow retry may look like a simple recovery, but it risks duplicate external side effects.
- Option B: The prompt cannot determine whether the payment API already processed the request after the timeout.
- Option C: Reducing model cost does not address the correctness risk of duplicating a payment action.

7

[→ Login to Clarify](#)

A production issue report says only, "Claude gave a bad answer." The current log record is:

level=error component=answer_service message="validation failed"

The team needs to diagnose future failures without storing full sensitive prompts. Which logging improvement is BEST?

- A.
Log only the user ID so support can contact the customer
- B. 
Log request ID, model, prompt version, status, and sanitized excerpts
- C.
Log the complete prompt and response for every production request
- D.
Log only the final rendered answer shown in the browser

Explanation:

Correct answer (B): Useful Claude debugging logs should capture the boundary and context needed for triage while avoiding unnecessary sensitive data exposure. Request ID, model version, prompt version, status, sanitized input or output excerpts, and validation details help determine whether the failure occurred in request handling, model output, or post-processing.

Why the other options are wrong:

- Option A: A user ID may help customer support, but it does not provide enough technical context to diagnose the failure boundary.
- Option C: Full prompts and responses can help debugging, but the scenario explicitly requires avoiding full sensitive prompt storage.
- Option D: The final browser output omits request metadata, raw response clues, and

validation details needed for root-cause analysis.

8



→ Login to Clarify

A regression appears only in the full application path. The trace summary is:

```
UI form -> request builder -> Claude API -> JSON parser -> database write
symptom: database row has null "priority"
api_status: 200
stored_raw_response: not captured
parser_version: changed yesterday
prompt_version: unchanged
```

What is the BEST next diagnostic step?

A.

Upgrade the model before collecting more traces

B.

Create a minimal repro that captures raw response before parsing



C.

Delete the parser change because all parser changes are unsafe

D.

Add more UI fields so Claude receives extra context

Explanation:

Correct answer (B): The failure could come from the model response, parser, or downstream database mapping, and the raw response was not captured. A minimal reproducible path that records the raw Claude response before parsing isolates whether ``priority`` was absent from the model output or lost in application processing, especially because the parser recently changed.

Why the other options are wrong:

- Option A: A model upgrade is premature because the trace lacks evidence that the model failed to produce the field.
- Option C: The parser change is suspicious, but deleting it without comparing raw output to parsed output skips diagnosis.
- Option D: Extra UI context may affect model behavior, but the prompt was unchanged and the immediate need is boundary isolation.

9 

[→ Login to Clarify](#)

A developer deploys a Claude-powered support triage service. After a small SDK wrapper change, every request fails before any model response is returned.

Log excerpt:

```
POST /messages
status: 400
error: "invalid_request"
detail: "messages[0].content must be an array of content blocks"
request_id: req_71a
model_response_body: null
```

What is the best next debugging action?

A.

Switch to a larger Claude model and rerun the failing requests.

B.

Inspect the client request construction and fix the message payload shape.



C.

Add a generic instruction telling Claude to return valid content blocks.

D.

Retry the same request with exponential backoff until it succeeds.

Explanation:

Correct answer (B): The failure occurs before Claude returns any model response, and the error explicitly identifies an invalid request payload. A 4xx-style invalid request should be debugged at the client request construction, authentication, parameters, or payload-format boundary before changing prompts, models, or retry policies.

Why the other options are wrong:

- Option A: Changing models may seem useful when an application fails, but the log shows the request is rejected before model output exists.
- Option C: Prompt instructions cannot fix a malformed API payload that is rejected before Claude processes the message.
- Option D: Backoff helps with transient errors, but this deterministic 400 error will continue until the request format is corrected.



A document extraction pipeline asks Claude for structured JSON and validates it before writing to a database. The same invoice fails on every run.

Artifact:

```
Required schema: {"invoice_id": string, "total_cents": integer, "currency": string}
Raw model output: {"invoice_id": "INV-8831", "total": "$42.10", "currency": "USD"}
Validator: missing required field "total_cents"; unexpected field "total"
Retries attempted: 3; same validation error each time
```

What is the best primary fix?

A.

Disable schema validation and let downstream code infer the total.

B.

Keep validation and align the prompt-output contract with the schema.



C.

Add more automatic retries because LLM outputs are non-deterministic.

D.

Accept both fields by loosening the schema for this invoice type.

Explanation:

Correct answer (B): The raw output violates the declared schema in a repeatable way, so the fix should preserve validation and repair the contract between the prompt, expected schema, parser, and downstream code. Blind retries or schema loosening would mask the defect rather than ensuring the application receives the required field in the required type.

Why the other options are wrong:

- Option A: Disabling validation may appear to improve throughput, but it hides a contract failure and risks incorrect downstream writes.
- Option C: Retries can help transient failures, but the artifact shows the same deterministic validation failure each time.
- Option D: Loosening the schema may be tempting, but the application explicitly requires `total_cents` as an integer field.